

Hackathon Prerequisites & Resources Guide

To work through the hackathon challenges, you'll need two things ready beforehand:

- **A coding setup** — for example, Claude Code, OpenCode, Codex, Cursor, or any other coding agent you're comfortable using.
- **Model / compute access** — either through hosted APIs, free model providers, cloud GPU runtimes, local hardware, or any other setup that works for you.

You're free to use any subscriptions, credits, tools, or infrastructure you already have. If you don't have a setup yet, this guide includes a few free options to get started.

The examples outlined are **not mandatory setups** — they're simply practical starting points in case you don't have your own setup.

Please have your coding environment and compute/model access set up and tested before the hackathon so you can start building right away.

Coding Setup Guide

If you already use **Claude Code, OpenCode, Codex, Cursor, or another coding agent** with your own model access or subscription, you are free to continue using that setup.

If you do not already have a setup, there are many workable combinations: a coding agent paired with a free model provider or gateway such as OpenRouter, another provider with a free tier, or a local model. You are encouraged to explore these options and pick what works best for you.

Example setup: The walkthrough below is just one possible setup. It uses Claude Code as the coding agent and OpenRouter to access a free model. You do not need to use this exact combination. You can instead use:

- A different coding agent with OpenRouter
- A different model provider (orcarouter, omnirouter, etc) that offers free access
- A local model running on your own machine
- Any other setup you are comfortable with and have tested beforehand

For the example below, have these ready before you begin:

- Claude Code installed on your machine
- An OpenRouter account
- A terminal you are comfortable using

Example: Claude Code + OpenRouter + a Free Model

Claude Code is simply the coding agent in this example. OpenRouter provides access to the model that Claude Code will use. The same general idea can be applied to other compatible coding agents and model providers.

1. Create an OpenRouter account and API key

Go to openrouter.ai and create an account. For this example, you can use models that are available for free, so you do not need to add credits.

Once you are logged in:

1. Go to Home.
2. Select API Keys from the left panel.
3. Create a new API key.
4. Copy the generated API key.

It should look something like:

```
sk-or-v1-...
```

Keep it private: Treat the API key like a password. Do not share it or commit it to your codebase.

2. Choose a free model

For this example, we will use:

```
nvidia/nemotron-3.5-lightning:free
```

You can browse currently available free text models here: [OpenRouter - Free Models](#).

Free-model availability can change. Pick a model you want to try and copy its model ID. You can always switch models later if one is rate-limited or does not work well with your coding agent.

3. Point Claude Code to OpenRouter

Run the following commands in the same terminal session where you plan to use Claude Code.

macOS / Linux / WSL

```
export ANTHROPIC_BASE_URL="https://openrouter.ai/api"
export ANTHROPIC_AUTH_TOKEN "<your-api-key>"
export ANTHROPIC_MODEL="nvidia/nemotron-3.5-lightning:free"
```

Replace `<your-api-key>` with the API key you copied from OpenRouter. If you selected another model, replace the value of `ANTHROPIC_MODEL` with that model ID.

Windows PowerShell

```
$env:ANTHROPIC_BASE_URL="https://openrouter.ai/api"
$env:ANTHROPIC_AUTH_TOKEN "<your-api-key>"
$env:ANTHROPIC_MODEL="nvidia/nemotron-3.5-lightning:free"
```

Note: These environment variables apply to the current terminal session. If you close the terminal and open a new one, you may need to set them again.

4. Verify that the setup works

Run a tiny one-off request:

```
claude -p "Reply with exactly: OK"
```

If the setup is working, you should receive:

```
OK
```

This confirms that Claude Code can make a request through the model you configured.

5. Confirm the selected model

Start Claude Code:

```
claude
```

Inside Claude Code, run:

```
/model
```

Confirm that the model you intended to use is selected. You can also run:

```
/status
```

Use this to confirm that Claude Code is routing through OpenRouter with the base URL and authentication token you configured.

Test Your Setup Before the Hackathon

Do not stop at the **OK** test. Try a few small coding tasks and make sure the setup is usable for you.

Free models can sometimes be rate-limited, temporarily unavailable, slower under demand, or more/less compatible with a particular coding agent. If one model is not working reliably:

- Open the free-model catalogue.
- Pick another free model.
- Update your selected model.
- Run the verification again.
- Try a small coding task.

Recommendation: have one or two backup model IDs or an alternative setup ready before the event.

Note

Different tools have different configuration steps, so if you choose another combination, follow that tool's documentation and test the complete flow beforehand. For reference, OpenRouter also documents integrations for [Claude Code](#) and [Codex CLI](#).

GPU & API Access for the Hackathon

You'll need compute to work through the hackathon challenges — whether that means running a model yourself or calling one through an API.

There are plenty of free options available, so we've put together a few that are easy to get started with:

- **Free GPU runtimes for running models yourself**
- **Hosted model APIs for accessing larger models without managing the compute**

These are only examples. You're free to use **any other compute setup, provider, credits, local hardware, or API access you already have**, as long as it works for what you're building.

Most teams will use a mix of approaches depending on the challenge.

Please set these up and test them before the hackathon so you can jump straight into building on the day.

Part 1 — Free GPU Runtimes

Use these when you want to run models yourself. Both options give you access to a T4 GPU in the browser, without requiring GPU setup on your own machine.

Option 1 — Kaggle Notebooks

1. Create an account at kaggle.com
2. Go to Settings → Phone Verification and verify your number (the GPU option stays locked until you do this).
3. Click Create → Notebook.
4. In the right sidebar, open Session options → Accelerator and choose GPU T4 x2.
5. Run this in the first cell to confirm it worked:

```
import torch
print(torch.cuda.is_available())    # should print True
```

You get 30 GPU hours per week, and the remaining amount is shown in the sidebar.

Option 2 — Google Colab

1. Open colab.research.google.com and sign in with a Google account.
2. Click New notebook.

3. Go to Runtime → Change runtime type, select T4 GPU, and click Save.

4. Run this in the first cell to confirm it worked:

```
import torch
print(torch.cuda.is_available())    # should print True
```

Two things to keep in mind

Check that the GPU is actually on. If the code above prints False, you're on a CPU and everything will run far slower. Recheck the accelerator setting.

Save your work as you go. Sessions can disconnect. Save results and checkpoints to /kaggle/working/ on Kaggle, or to Google Drive on Colab:

```
from google.colab import drive
drive.mount('/content/drive')
```

Part 2 — Hosted Model APIs

Use these when you want to call a large hosted model without running it yourself. No GPU is required.

Option 1 — NVIDIA NIM

A catalog of 100+ hosted models, including vision models. Free, with no credit card required.

1. Go to build.nvidia.com and create an account.
2. Verify your phone number when prompted — API access stays locked until you do.
3. Browse the vision models at build.nvidia.com/explore/vision
4. Open any model page and click Get API Key. Copy the key immediately — you usually only see it once.
5. Call it using the OpenAI SDK, just with a different base URL:

```
from openai import OpenAI

client = OpenAI(
    base_url="https://integrate.api.nvidia.com/v1",
    api_key="nvapi-YOUR-KEY-HERE"
)

response = client.chat.completions.create(
    model="meta/llama-3.2-90b-vision-instruct",
    messages=[{"role": "user", "content": "Describe this scene."}]
)
print(response.choices[0].message.content)
```

Good to know: the free tier allows around 40 requests per minute. Any tool or library that already speaks the OpenAI API format will work here by changing the base URL and model name.

Option 2 — Gemini API (free tier)

Useful because Gemini accepts video and images directly. Free, with no credit card required.

1. Go to aistudio.google.com and sign in with a Google account.
2. Click Get API key → Create API key.
3. Install the SDK:

```
pip install google-genai
```

4. Make your first call:

```
from google import genai

client = genai.Client(api_key="YOUR-KEY-HERE")

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents="Say hello."
)
print(response.text)
```

Good to know

- The free tier covers Flash and Flash-Lite models only. Pro models now require billing, so build with Flash from the start rather than prototyping on Pro and hitting a wall.
- Limits are per project, not per key. Creating extra API keys inside the same project will not give you more quota.
- Your exact limits are shown in AI Studio. Check there rather than relying on numbers from blog posts, since Google changes them without notice.
- If you get a 429 error, you've hit a rate limit. Add a short wait and retry rather than hammering the endpoint.

Before you arrive

- ☐ Kaggle account created and phone-verified
- ☐ Colab opened at least once
- ☐ NVIDIA API key generated and saved
- ☐ Gemini API key generated and saved
- ☐ One test call made successfully with each

Choose the setup that works for you, test it beforehand, and come ready to build!